



PEC4: Introducción al aprendizaje computacional

Presentación

Cuarta PEC del curso de Inteligencia Artificial

Competencias

En esta PEC se trabajarán las siguientes competencias:

Competencias de grado:

- Capacidad de analizar un problema con el nivel de abstracción adecuado a cada situación y aplicar las habilidades y conocimientos adquiridos para abordarlo y solucionarlo.

Competencias específicas:

- Conocer los diferentes aspectos del aprendizaje computacional (aprendizaje supervisado, no supervisado y por refuerzo).
- Conocer los fundamentos teóricos de los métodos más representativos.
- Conocer el tratamiento de los conjuntos de datos para la correcta validación de los sistemas de aprendizaje.

Objetivos

Esta PEC pretende evaluar diferentes aspectos de aprendizaje supervisado y no supervisado.

Descripción de la PEC a realizar

Pregunta 1 (4 puntos)

Una agencia espacial ha estado realizando un conjunto de observaciones de distintos planetas y quieren realizar un modelo que permita predecir si uno de ellos es habitable (Y) o no lo es (N) en base a las siguientes características:

- Medida: Grande (B), Pequeño (S)
- Distancia de su órbita al sol: Cerca (N), Lejos (F)
- Temperatura: Baja (L), Media (M), Alta (H).

La tabla con las observaciones es la siguiente:



	Medida	Distancia órbita	Temperatura	Habitable
1	S	N	L	Y
2	S	N	M	Y
3	S	F	H	Y
4	S	F	L	N
5	B	F	H	N
6	B	F	L	N
7	B	F	L	N
8	B	F	M	Y
9	S	N	M	Y
10	S	N	L	Y
11	B	N	M	Y
12	B	N	H	N
13	S	F	L	N
14	B	N	M	Y
15	B	N	H	N
16	S	F	L	N



En este ejercicio, debéis construir un árbol de decisión con las tres características que tenemos en la tabla anterior, siendo Habitable el objeto de clasificación.

Solución

Para construir el árbol seguimos un proceso iterativo, en el que calculamos la bondad de los atributos. Es decir, lo bien clasificada que nos queda la clase vuelo según la característica que elegimos. En la tabla vemos el cálculo:

	Habitable		Bondad	
Medida	Y	N		
S	5	3		
B	3	5	$(5+5)/16=$	0,625
Distancia				
N	6	2		
F	2	6	$(6+6)/16=$	0,75
Temperatura				
L	2	5		



M	5	0	
H	1	3	$(5+5+3)/16= 0,8125$

Podemos ver que la temperatura es la característica que mejor separa la clase objetivo (habitable). A continuación, deberemos evaluar las tres ramas que obtenemos de esta partición. Construiremos el árbol en profundidad y luego procederemos a calcular las dos ramas restantes.

Si elegimos **temperatura = L** como rama izquierda:

	Medida	Distancia órbita	Habitable
1	S	N	Y
4	S	F	N
6	B	F	N
7	B	F	N
10	S	N	Y
13	S	F	N
16	S	F	N

El cálculo de la bondad es el siguiente:



L	Habitable		Bondad	
Medida	Y	N		
S	2	3		
B	2	0	$(2+3)/7=$	0,71428
Distancia				
N	2	0		
F	0	5	$(2+5)/7=$	1

Podemos observar que con la característica distancia tenemos una bondad de 1. No necesitamos seguir desarrollando esta rama. Por otra parte tenemos que con la **temperatura media** (M), no es necesario desarrollar más niveles del árbol. Vamos a desarrollar la rama derecha con la que evaluaremos la **bondad = temperatura alta** (H)

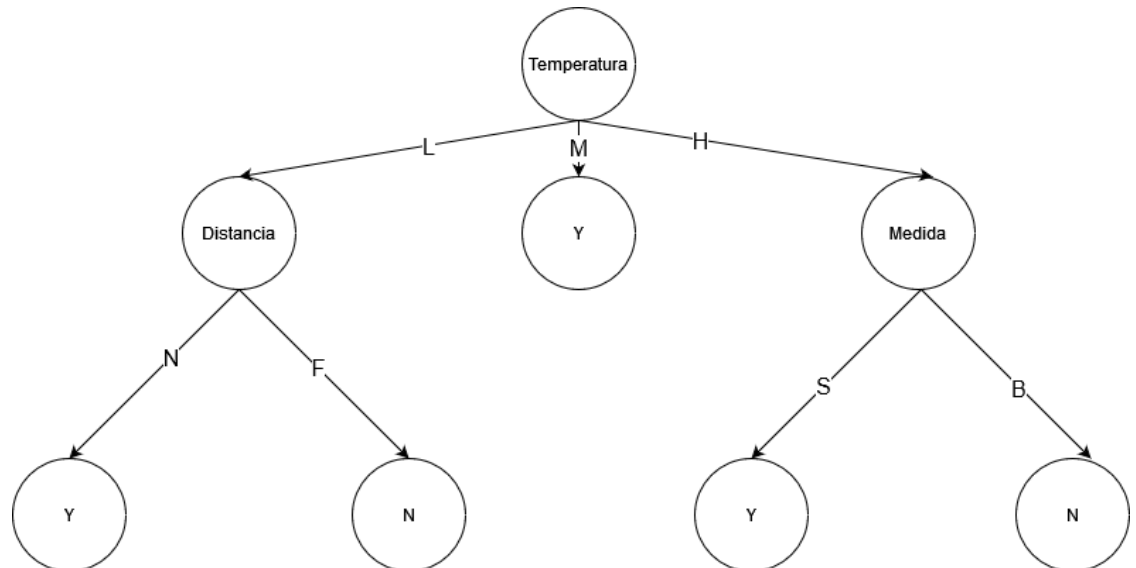
	Medida	Distancia órbita	Habitable
3	S	F	Y
5	B	F	N
12	B	N	N
15	B	N	N

El cálculo de la bondad es el siguiente:



H	Habitable		Bondad
Medida	Y	N	
S	1	0	
B	0	3	$(1+3)/4 = 1$
Distancia			
N	0	2	
F	1	1	$(2+1)/4 = 0,75$

Podemos observar que con la característica medida tenemos una bondad de 1. El árbol resultante es el siguiente:



Ejercicio 2 (4 puntos)

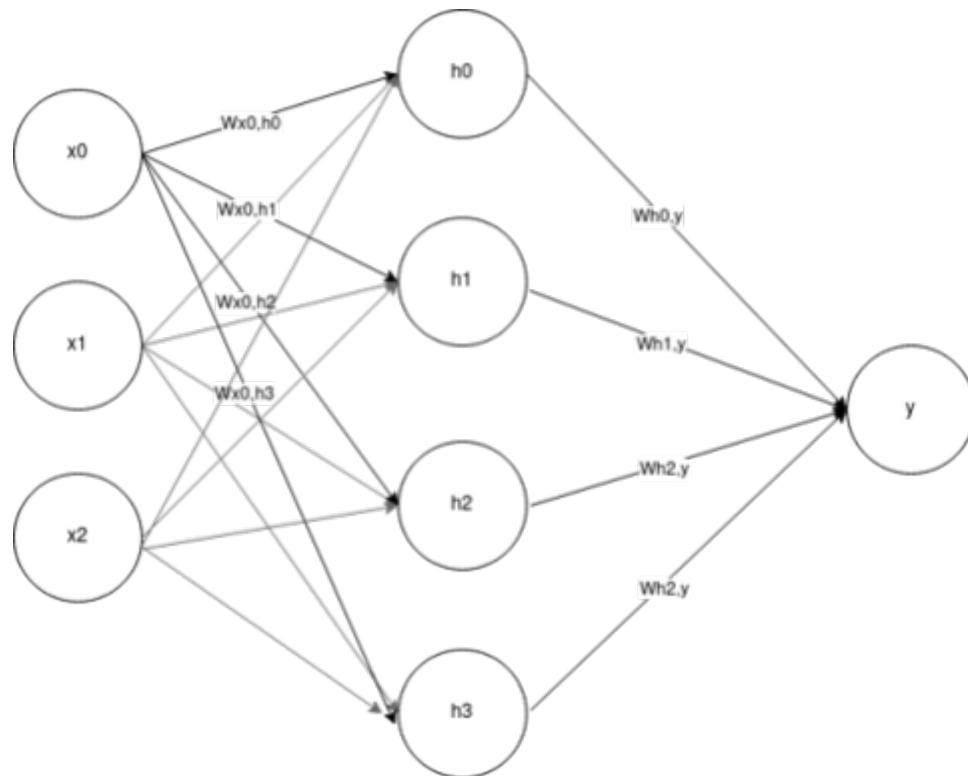
Tal como ya sabemos, el uso de una red neuronal puede ser más adecuado cuando los atributos de nuestro problema son numéricos y no categóricos como en el ejercicio anterior. Por este motivo, hemos diseñado y entrenado una red neuronal a partir de las observaciones de tres sensores. Los sensores pueden tomar valores enteros entre 0 y 10. La salida de la red puede ser 0 o 1.



Queremos comprobar la validez de nuestro clasificador con el siguiente conjunto de datos:

Sensor1	Sensor2	Sensor3	Clase
6	8	2	1
9	7	1	0
1	4	2	0
3	10	1	0
1	3	1	1
6	7	1	1
7	9	3	1
9	0	1	0
0	4	3	1
5	2	1	0
5	7	2	0
0	4	2	1
1	10	3	1
7	3	2	0
6	8	2	1

Dada la siguiente red neuronal ya entrenada donde las neuronas x_0 , x_1 e x_2 forman la capa de entrada en la que introduciremos los datos del Sensor1, el Sensor2 y el Sensor3 respectivamente.



Donde:

$h_0 = w_{x_0,h_0}x_0 + w_{x_1,h_0}x_1 + w_{x_2,h_0}x_2$
$h_1 = w_{x_0,h_1}x_0 + w_{x_1,h_1}x_1 + w_{x_2,h_1}x_2$
$h_2 = w_{x_0,h_2}x_0 + w_{x_1,h_2}x_1 + w_{x_2,h_2}x_2$
$h_3 = w_{x_0,h_3}x_0 + w_{x_1,h_3}x_1 + w_{x_2,h_3}x_2$
$Y = w_{h_0,y}h_0 + w_{h_1,y}h_1 + w_{h_2,y}h_2 + w_{h_3,y}h_3$

Los pesos asociados a cada una de las conexiones neuronales están definidos en la siguiente tabla:

W	h_0	h_1	h_2	h_3
x_0	1	-0,5	0,5	1
x_1	-1	1	1	-0,5
x_2	1	-0,5	-0,25	1
z	-1	-0,5	1	-0,25



La función de activación más utilizada en la capa oculta es la ReLU y en la capa de salida se aplica la siguiente función *heaviside*:

$$heaviside(x) = \begin{cases} 0, & \text{if } x \leq 0 \\ 1, & \text{if } x > 0 \end{cases}$$

Calcula la salida de la red para cada observación, la matriz de confusión de este clasificador y la métrica F1. Para calcular la matriz de confusión y la métrica F1 consideraremos 1 como el valor positivo y 0 como el valor negativo.

Solución

En la siguiente tabla tenéis todos los cálculos necesarios para encontrar la solución:

S1	S2	S3	h0	h1	h2	h3	y0	Salida	Clase
6	8	2	0	4	10,5	4	7,5	1	1
9	7	1	3	2	11,25	6,5	5,625	1	0
1	4	2	0	2,5	4	1	2,5	1	0
3	10	1	0	8	11,25	0	7,25	1	0
1	3	1	0	2	3,25	0,5	2,125	1	1
6	7	1	0	3,5	9,75	3,5	7,125	1	1
7	9	3	1	4	11,75	5,5	7,375	1	1
9	0	1	10	0	4,25	10	-8,25	0	0
0	4	3	0	2,5	3,25	1	1,75	1	1
5	2	1	4	0	4,25	5	-1	0	0



5	7	2	0	3,5	9	3,5	6,375	1	0
0	4	2	0	3	3,5	0	2	1	1
1	10	3	0	8	9,75	0	5,75	1	1
7	3	2	6	0	6	7,5	-1,875	0	0
6	8	2	0	4	10,5	4	7,5	1	1
Activació			ReLU	ReLU	ReLU	ReLU		Heaviside	

Ahora debemos obtener las métricas, en primer lugar la matriz de confusión:

		Real	
		Positiva	Negativa
Predicción	Positiva	8	4
	Negativa	0	3

Precisión = $tp/(tp+fp) = 0,667$

Sensibilidad = $tp/(tp+fn) = 1$

F1 = $2(Precisión \times Sensibilidad)/(Precisión + Sensibilidad) = 0,800$

Pregunta 3 (2 puntos)

El notebook de Python adjunto genera un conjunto de 400 puntos en un espacio 2D. Además, podréis encontrar implementado el algoritmo de kNN excepto la función de distancia.

Debéis realizar las siguientes tareas:

- Implementar la función de distancia euclídea.



- b) Calcular la matriz de confusión y la métrica XXX. Podéis usar la librería [scikit](#) para simplificar la programación, aunque no es obligatorio.
- c) Repetir el cálculo de la métrica para 1, 2, 4 y 5 vecinos. ¿Para el caso que nos ocupa, con qué valor obtenemos los mejores resultados?

Solución

```
In [1]: # librerías para generar los datos del problema
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

from math import sqrt # función para calcular la raíz cuadrada de una variable
import matplotlib.pyplot as plt # librería para dibujar gráficos
```

Datos

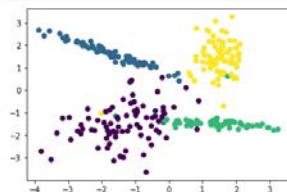
Generamos un problema de clasificación de datos en el que hay 4 clases diferentes. El conjunto de datos consta de 400 muestras. nos quedamos el 75% (X_train) para entrenar el modelo y el 25% (X_test) restante nos servirá para evaluarlo.

Notad que cada muestra del conjunto de datos tiene dos dimensiones como se puede observar en el gráfico.

```
In [2]: # Nota: Este código no se puede cambiar
X, y = make_classification(n_samples=400, n_features=2, n_informative=2, n_redundant=0,
                           n_repeated=0, n_classes=4, n_clusters_per_class=1, class_sep=1.5,
                           random_state=10)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=10)

# Visualizamos los datos de entrenamiento
plt.scatter(X_train[:,0], X_train[:,1], c=y_train);
```



Cálculos

```
In [9]: # Función distancia euclídea
def euclidean_distance(row1, row2):
    distance = 0.0
    for i in range(len(row1)):
        distance += (row1[i] - row2[i])**2
    return sqrt(distance)

# Función para encontrar los vecinos más cercanos
def get_neighbors(train, labels, test_row, num_neighbors):
    distances = list()
    for train_row in train:
        dist = euclidean_distance(test_row, train_row)
        distances.append(dist)

    ordered_labels = [x for _, x in sorted(zip(distances, labels))]
    neighbors = list()
    for i in range(num_neighbors):
        neighbors.append(ordered_labels[i])
    return neighbors

# Función para predecir la clase de un conjunto de datos:
# train: conjunto de entrenamiento, con el que comparamos. Matriz con número
# de filas = número de elementos en el conjunto y 2 columnas.
#
# Labels: etiquetas del conjunto de entrenamiento. Lista con número
# de filas = número de elementos en el conjunto
#
# test: conjunto de evaluación. Matriz con número
# de filas = número de elementos en el conjunto y 2 columnas.
#
# num_neighbors: Vecinos a considerar
def predict_classification(train, labels, test, num_neighbors):
    predictions = list()
    for test_row in test:
        neighbors = get_neighbors(train, labels, test_row, num_neighbors)
        prediction = max(set(neighbors), key=neighbors.count)
        predictions.append(prediction)
    return predictions
```



Clasificación

Usamos el 25% que hemos reservado para evaluar el modelo lo evaluamos con 3 vecinos y obtenemos la predicción.

```
In [4]: from sklearn.metrics import confusion_matrix

for i in range(1,5):
    prediction = predict_classification(X_train, y_train, X_test, i)
    print("Resultados k = ", i)
    cf = confusion_matrix(y_test, prediction)
    print(cf)
    tp = 0
    for j in range(cf.shape[0]):
        tp += cf[j][j]
    print("True Positives: ", tp)

Resultados k = 1
[[17  0  1  0]
 [ 0 28  0  0]
 [ 0  0 32  0]
 [ 2  0  0 20]]
True Positives: 97
Resultados k = 2
[[17  0  1  0]
 [ 2 26  0  0]
 [ 0  0 32  0]
 [ 2  0  0 20]]
True Positives: 95
Resultados k = 3
[[17  0  1  0]
 [ 1 27  0  0]
 [ 0  0 32  0]
 [ 2  0  0 20]]
True Positives: 96
Resultados k = 4
[[17  0  1  0]
 [ 0 27  0  1]
 [ 0  0 32  0]
 [ 2  0  0 20]]
True Positives: 96
```

Recursos

Para hacer esta PEC el material imprescindible es el módulo 5. Para ejecutar los notebooks en Python del ejercicio se recomienda usar Google Colab que es gratuito (<https://colab.research.google.com>) y provee de todo lo necesario para su realización.

Criterios de valoración

Las puntuaciones se muestran en cada pregunta del enunciado.

Formato y fecha de entrega

Para dudas y aclaraciones sobre el enunciado, dirigiros al consultor responsable del aula.

Hay que entregar la solución en un archivo PDF usando una de las plantillas entregadas conjuntamente con este enunciado. Adjuntar el fichero a un mensaje en el apartado Entrega y Registro de EC (REC).

El nombre del archivo debe ser *Apellidos_Nombre_IA_PEC4* con la extensión .pdf (PDF).

La fecha límite de entrega es el: **19/12/2021** (a las 24 horas).

Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.



Nota: Propiedad intelectual

A menudo es inevitable, al producir una obra multimedia, hacer uso de recursos creados por terceras personas. Es por tanto comprensible hacerlo en el marco de una práctica de los estudios de Informática, siempre que se documente claramente y no suponga plagio en la práctica.

Por lo tanto, al presentar una práctica que haga uso de recursos ajenos, se presentará junto con ella un documento en el que se detallen todos ellos, especificando el nombre de cada recurso, su autor, el lugar donde se obtuvo y el su estatus legal: si la obra está protegida por copyright o se acoge a alguna otra licencia de uso (Creative Commons, licencia GNU, GPL ...).

El estudiante deberá asegurarse de que la licencia que sea no impide específicamente su uso en el marco de la práctica. En caso de no encontrar la información correspondiente deberá asumir que la obra está protegida por copyright.

Deberán, además, adjuntar los archivos originales cuando las obras utilizadas sean digitales, y su código fuente..